

石器时代之宠物综述

Rode (QQ:81881788)

2026 年 4 月 16 日

摘要

本文以 `gmsv` 源码为最高优先级证据，系统整理《石器时代》宠物成长、转生、融合、蛋喂药、孵化及其向战斗能力映射的数值机制。全文首先统一宠物成长档、蛋基础成长、孵化后成长与战斗输出之间的术语口径，随后依次讨论 Rank 判定与成长随机、转生能力值的形成与分配、融合成蛋与喂药压缩的离散规则，以及成长差异如何经由面板与工作值层映射到战斗中的攻、防、敏表现。研究表明，宠物培养问题并不是连续意义上的“总和越高越好”，而是一类同时受多次取整、单项上限、总和压缩与比例回分配约束的离散优化问题。玩家经验中的分歧，主要来自规则生效位置、压缩边界与随机阶段的混同。本文据此将宠物系统整理为一条由源码证据、数学表达与培养解释相互支撑的分析链，为后续的喂药搜索、养成比较与战斗目标反推提供稳定口径。

关键词：石器时代；宠物成长；蛋喂药；源码分析；离散优化

目录

第一章 绪论	1
1.1 研究背景	1
1.2 研究问题	1
1.3 研究方法与证据标准	1
1.4 主要贡献	2
1.5 全文结构	2
第二章 研究对象、证据口径与方法框架	3
2.1 研究对象与版本边界	3
2.2 符号体系与术语映射	3
2.3 离散运算的统一表示	4
2.4 方法框架	5
第三章 宠物基础成长与显示属性映射	6
3.1 成长档与显示属性映射的基本结构	6
3.2 初始化随机与 Rank 门槛	7
3.3 成长约束的两种来源	9
3.4 蛋基础成长与孵化后成长的区分	10
第四章 宠物养成链：转生、融合与蛋喂养	11
4.1 转生作为成长档重构	11
4.2 融合作为逐步截断变换	13
4.3 喂药与孵化的两级压缩结构	14
4.4 孵化结果的解释口径	17
4.5 养成流水线的统一表示	17
第五章 宠物养成到战斗能力的映射	18
5.1 从 (V, S, T, D) 到战斗输出的映射	18
5.2 从转生、融合到战斗能力的长期收益	18
5.3 从蛋喂养到战斗目标的定向优化	19
5.4 宠物忠诚失控 (nono) 与动作改写	19
5.5 不同培养目标下的宠物类型	21

第六章 离散优化模型、案例与策略含义	22
6.1 优化问题的形式化	22
6.2 蛋喂养最优化	22
6.3 战斗目标反推	23
6.4 转生与融合策略比较	23
6.5 案例研究	24
第七章 结论	25
7.1 主要研究结论	25
7.2 对培养经验的修正	25
7.3 研究局限与后续工作	26
附录 A 符号、术语与字段对照	27
附录 B 公式总表	29
附录 C 源码索引	31

第一章 绪论

1.1 研究背景

《石器时代》的宠物系统同时包含成长随机、转生重分配、融合成蛋、蛋喂药与孵化压缩等多层离散规则，因此围绕“高成长宠如何形成”“哪些喂药方案更优”“转生与融合各自改变了什么”的经验判断，往往带有明显的版本依赖与口径差异。实践中的分歧通常并不来自结论本身，而来自对规则入口的混同：或将蛋阶段的基础成长与孵化后成长视为同一量，或忽略逐层向下取整与总和压缩，或把高等级战斗表现直接回溯为对单一成长维度的线性判断。

与战斗系统相比，宠物养成的复杂性更集中地体现为“长期累计”与“离散边界”。同样一组素材，在低于等级门槛时会先经历折扣；同样一组药值，在折算、50 工作值压缩与 150 孵化压缩之后可能得到完全不同的最终成长；同样一只宠物，在普通宠与转生宠两套 Rank 表下又会进入不同的升级随机区间。由此，宠物培养问题不宜仅以经验总结处理，而需要回到源码，重新界定每一层规则的输入、输出与取整位置。

1.2 研究问题

本文围绕三个互相关联的问题展开。第一，宠物从基础成长到高等级面板的生成过程，能否被整理为一套清晰、可复核的离散模型。第二，转生、融合、蛋喂药与孵化分别在哪一层改变成长档，并通过哪些压缩与随机环节影响最终结果。第三，宠物培养差异如何经由显示面板、速度相关输出与动作改写机制转化为实战中的攻防与稳定性差异。

上述问题的共同点，在于都要求区分“数值写入发生在何处”与“结果表现出现于何处”。若不能把成长阶段、蛋阶段、孵化阶段与战斗映射阶段明确分开，则关于最优喂药、转生收益或培养方向的结论，就容易被写成缺乏边界条件的经验口号。

1.3 研究方法 with 证据标准

本文以当前持有的 gmsv 源码为最高优先级证据，并以项目 sa_pet_assistant 中已与源码核对的实现作为辅助复核环境；该项目的公开仓库为 https://github.com/Rode880919/Stone_age_tools。在方法上，本文依次完成四項工作：其一，核对宠物成长、

转生、融合、喂药与孵化的关键数据入口；其二，将分散在不同阶段的数值统一到同一套术语与符号体系中；其三，把多次取整、总和压缩与成长映射写成可分析的离散表达；其四，在此基础上讨论培养目标、代理目标与最优方案的结构差异。

在结论表述上，本文采取保守口径。凡当前源码中尚未直接验证、可能因编译路径而改变，或需要额外日志复核的机制，均只给出条件性结论，而不以玩家经验补足数学表达。

1.4 主要贡献

本文的主要贡献可概括为四点。第一，建立了宠物成长、蛋成长与孵化后成长之间的统一术语口径，降低不同玩家习惯记法之间的歧义。第二，系统还原了转生、融合、喂药与孵化的离散规则，说明多次取整与压缩为何会改变看似直观的“总和优势”。第三，将宠物培养与战斗输出连接为一条可计算的映射链，解释成长维度为何会在高等级阶段表现为截然不同的培养方向。第四，把典型培养问题写成显式优化问题，从而为喂药搜索和养成比较提供更稳定的数学框架。

1.5 全文结构

全文结构如下。第 2 章给出本文使用的对象边界、符号体系与方法框架。第 3 章讨论宠物基础成长、Rank 表与升级随机。第 4 章分析转生、融合、蛋喂药与孵化的完整流水线。第 5 章说明成长差异如何映射到显示面板、速度与动作结果。第 6 章在前述机制基础上建立离散优化模型，并给出代表性的培养案例。第 7 章总结全文结论、适用边界与后续问题。

本文的论证顺序遵循“先定义对象，再还原流程，最后讨论优化”的路径。只有先明确成长如何生成、压缩在何处发生、战斗输出又如何由成长传递，关于“培养何种宠物更优”的判断才具有可检验性。

第二章 研究对象、证据口径与方法框架

本章给出本文共同采用的研究对象、符号体系与证据口径，并统一说明成长随机、取整与上下限压缩在分析中的表示方式。后文关于成长生成、蛋喂药与战斗映射的推导，均以本章所确立的对象边界与离散规则为前提。

2.1 研究对象与版本边界

本文的研究对象包括四类：普通宠物、转生宠物、融合蛋，以及由宠物成长进一步映射出的战斗输出。本文所称“战斗输出”，仅限宠物成长进入显示面板、速度相关结果与宠物动作结果的过程，并不展开完整战斗系统。忠诚失控等机制之所以纳入分析，是因为它们会改变成长向实际输出的最终映射。

本文的主要证据来源是当前持有的 `gmsv` 代码树及其配套整理结果。对于已经核清的成长规则，正文直接给出离散表达；对于尚未完全确认的边界，则保持条件性描述。因此，本文构建的是一个在当前源码与当前编译口径下自洽的宠物数值模型，而不是对所有服务器实现的并行概括。

2.2 符号体系与术语映射

本文采用表 2.1 与表 2.2 所示的统一符号体系。前者用于描述成长、转生与孵化阶段的数值对象，后者用于描述从宠物成长到战斗输出的映射层。完整记号与术语对照见附录 A。

表 2.1: 成长、转生与孵化阶段的核心记号

记号	含义	说明
V, S, T, D	四项成长率	分别对应生命、攻击、防御、敏捷方向的成长来源
g	基础成长档	正文主记号；玩家口径常写作 (V, S, T, D)
$VSTD$	四项成长和	定义为 $V + S + T + D$
L_v	等级	用于区分一级、成长中与高等级阶段
b	蛋基础成长	融合完成后、喂药前或喂药后的蛋面值
h	孵化后成长	经孵化压缩得到的成长，不含出生后随机加点
A	转生能力值	转生重分配之前的中间能力量
$A_{\max}$	转生能力上限	默认分析中分别对应 150 或 200 的上限裁切

表 2.2: 宠物到战斗输出映射阶段的核心记号

记号	含义	说明
$r = (V_r, S_r, T_r, D_r)$	高等级原始累计点	表示成长经过随机与升级累计后的四维结果
HP, ATK, DEF, SPD	宠物显示四围	分别表示生命、攻击、防御、敏捷的面板值
Q	速度相关输出量	宠物行动顺序与速度相关结果的概括记号
a	本回合固定忠诚值	宠物忠诚检查直接读取的稳定忠诚值
$\Psi(\cdot)$	成长到输出的复合映射	表示成长、面板、速度与动作结果的综合映射
$\Omega(\cdot)$	忠诚改写后的动作输出	表示原指令与改写结果共同决定的实际动作

术语映射采用三层并行规则：源码层保留关键字段与动作入口；数学层使用 g 、 b 、 h 、 Ψ 等记号表达；语义层则使用“基础成长档”“蛋基础成长”“孵化后成长”“固定忠诚值”“速度相关输出”等中文术语解释其作用。三者之间保持稳定对应，是后文避免概念滑移的前提。

2.3 离散运算的统一表示

宠物系统中的争议大多来源于随机、截断与总和压缩三类离散操作的位置。源码中的随机宏输出恒为整数，因此任何随机过程在进入后续阶段前都已回到整数域。与此同

时，许多比例修正虽然写成乘法，但一旦重新写回整型变量，就会表现为向下截断而非四舍五入。例如，

$$x' = \lfloor 0.8x \rfloor \quad (2.1)$$

与

$$y' = \lfloor 0.6y \rfloor \quad (2.2)$$

在数值意义上都应理解为“先缩放，再丢弃小数部分”。

为统一表示上下限约束，本文使用裁切算子

$$\text{clip}_{[a,b]}(z) = \min\{\max\{z, a\}, b\}. \quad (2.3)$$

该写法能够将成长主体与压缩边界分离表述。转生能力上限、50 工作值压缩与 150 孵化压缩，均可在保留主体结构的同时，用同一类算子统一表达其边界效应。

需要特别指出的是，宠物系统中的平台、跳变与局部反直觉现象，通常不是由单一公式造成，而是由“折扣 → 取整 → 压缩 → 再分配”这一串联过程共同引入。后文所有优化讨论，均以前述离散结构为基础。

2.4 方法框架

本文的方法框架由四个相互衔接的步骤构成。第一步是机制核对，即逐段确认成长、转生、融合、喂药与孵化的关键入口。第二步是符号化整理，即把不同阶段的数值对象统一到同一套记号体系中。第三步是离散表达，即将成长随机、总和压缩与映射过程写成可分析的分段表达。第四步是综合解释，即在此基础上讨论培养目标、案例比较与离散优化结构。

在这一框架中，源码始终是最高优先级证据，工具实现只用于复核已经由源码支持的结论。也正因为如此，正文中的任何高层概括都必须能够回指到具体的成长入口、压缩规则或动作改写链，而不能脱离这些数值位置独立成立。

第三章 宠物基础成长与显示属性映射

本章从源码证据出发，重建宠物基础成长进入可见面板的映射过程。分析重点不在于罗列单项成长率，而在于说明成长档、随机初始化、等级推进与显示公式之间的层次关系。只有先澄清这一基础映射，后文关于转生、融合、喂药与战斗表现的讨论才可能建立在统一的变量口径之上。

3.1 成长档与显示属性映射的基本结构

在本文中，宠物的基础成长方向由四维向量

$$\mathbf{g} = (V, S, T, D) \quad (3.1)$$

表示，其中四个分量分别对应生命、攻击、防御与敏捷方向的成长率。对应的总和记为

$$VSTD = V + S + T + D. \quad (3.2)$$

$VSTD$ 虽然能够概括成长总量，却不足以唯一表征宠物。对于相同总和，只要四维分配不同，成长方向与实战能力便可能出现明显差异。本文因此在讨论中同时保留总和视角与分量视角。

宠物原始成长点数到显示属性的换算为

$$HP = \left\lfloor \frac{4V_r + S_r + T_r + D_r}{100} \right\rfloor, \quad (3.3)$$

$$ATK = \left\lfloor \frac{S_r + 0.1T_r + 0.1V_r + 0.05D_r}{100} \right\rfloor, \quad (3.4)$$

$$DEF = \left\lfloor \frac{T_r + 0.1S_r + 0.1V_r + 0.05D_r}{100} \right\rfloor, \quad (3.5)$$

$$SPD = \left\lfloor \frac{D_r}{100} \right\rfloor, \quad (3.6)$$

其中 (V_r, S_r, T_r, D_r) 为原始累计点数。式 (3.6) 表明四项成长并非彼此独立地映射到四项面板属性： V 主要进入生命，同时少量进入攻击与防御； D 主要进入敏捷，同时少量进入攻击与防御。该映射是带交叉项的线性变换叠加整数截断，因此具有两项性质：(1) 多对一，即不同原始点数组合在向下取整后可产生相同显示属性；(2) 层级性，即面板属性由随机过程累积得到的 (V_r, S_r, T_r, D_r) 决定，而非由基础成长率 (V, S, T, D) 直接决定。由显示属性反推成长率时，只能得到候选集合而不能唯一还原。

3.2 初始化随机与 Rank 门槛

3.2.1 一级初始化的离散来源

普通宠物的一级初始化包含三步。第一步是四维独立的 ± 2 波动：

$$\varepsilon_i \in \{-2, -1, 0, 1, 2\}, \quad g_i^{(s)} = g_i + \varepsilon_i, \quad (3.7)$$

其中 $g_i^{(s)}$ 表示真正进入成长档的“生效成长率”。第二步依据波动后的四维总和判定 Rank，而不是依据基础输入的 $VSTD$ 判定。第三步是在出生时进行一次总计 10 点的随机分配，每次随机给四维中的某一项加 1。当前配套的宠物成长模拟，为与既有模拟口径保持一致，在此额外采用了“单项最多额外得到 4 点”的限制；需要强调的是，这一限制属于工具侧模拟设定，并非本文已直接从服务器源码验证出的硬编码上限。若按“10 次独立、每次以 1/4 概率落到某一维”的无上限模型考察，对预先指定的一项有

$$X \sim \text{Binomial}(10, 1/4), \quad \Pr(X \geq 5) = \sum_{k=5}^{10} \binom{10}{k} (1/4)^k (3/4)^{10-k} \approx 7.81\%. \quad (3.8)$$

进一步地， $\Pr(X \geq 6) \approx 1.98\%$ ， $\Pr(X \geq 7) \approx 0.35\%$ 。在无上限随机模型下，“单项超过 4 点”并非不可能事件，只是对预先指定的某一维而言概率较低。若将出生时随机加点记作 e_i ，初始化参数记作 $INITNUM$ ，则一级原始点数满足

$$g_{r,i}^{(1)} = INITNUM \cdot (g_i^{(s)} + e_i). \quad (3.9)$$

这说明一级面板已经混合了基础成长率、 ± 2 波动、出生随机加点与初始化尺度四类信息，出生时刻的显示属性本身就不是单一成长档的直接呈现。

3.2.2 普通宠与转生宠的 Rank 门槛差异

Rank 是宠物成长系统中的关键中介变量。普通宠物（0 转宠）与转生宠使用不同的 Rank 判定门槛表。

普通宠物的 Rank 由波动后的 $VSTD$ 判定，分界如表 3.1 所示。

表 3.1: 普通宠物（0 转宠）的 Rank 判定表

波动后 $VSTD$	Rank
≥ 100	0
≥ 95	1
≥ 90	2
≥ 85	3
≥ 80	4
< 80	5

对于转生宠，Rank 判定门槛会发生变化，其分界如表 3.2 所示。

表 3.2: 转生宠的 Rank 判定表

波动后 $VSTD$	Rank
≥ 130	0
≥ 100	1
≥ 95	2
≥ 85	3
≥ 80	4
< 80	5

两表的关键差异在于 Rank 0 门槛由 100 提高到 130，从而改变了高成长转生宠的判定标准。Rank 判定对象始终是波动后的生效成长总和，但门槛表会随宠物是否已经转生而变化。首转计算原宠 Rank 时仍使用普通表；转生后的新成长率重新进入成长流程时，则按转生宠 Rank 表判定（详见第 4 章）。

Rank 一经确定，就会进入升级随机的倍率表。当前实现中，不同 Rank 对应的随机倍率区间如表 3.3 所示。

表 3.3: 不同 Rank 的升级倍率范围

Rank	$fRand$ 对应整数区间
0	$[450, 500] \times 0.01$
1	$[470, 520] \times 0.01$
2	$[490, 540] \times 0.01$
3	$[510, 560] \times 0.01$
4	$[530, 580] \times 0.01$
5	$[550, 600] \times 0.01$

由此可见，Rank 并非单纯的标签变量，而是后续升级随机参数的入口。同一只基础宠物若因 ± 2 波动跨过 Rank 边界，其后续升级路径将整体改变。

3.2.3 两类成长流程的共同部分与差异

0 转宠与转生宠真正不同的是 Rank 判定门槛，而非整个成长引擎。一级初始化公式、出生 10 点随机、升级时的 10 点随机、 $fRand$ 倍率表与显示公式均相同，变化的仅是 Rank 表。两者的完整对照见表 3.4。

表 3.4: 0 转宠与转生宠的成长口径对照

比较项目	0 转宠	转生宠
适用对象	普通捕捉宠、普通孵化宠、尚未进入转生路径的宠物	已完成转生并重新进入成长流程的宠物
Rank 判定对象	波动后的生效成长总和 $\sum_i g_i^{(s)}$	波动后的生效成长总和 $\sum_i g_i^{(s)}$
Rank 门槛	使用表 3.1: Rank 0 门槛为 100	使用表 3.2: Rank 0 门槛提高到 130
一级出生 10 点随机	总计 10 点; 当前工具模拟默认额外采用单项最多 4 点的限制	同左
升级时 10 点随机	总计 10 点, 不设单项 4 点限制	同左
$fRand$ 区间表	使用表 3.3	与 0 转宠相同, 仍使用表 3.3
显示属性换算	使用式 (3.6)	与 0 转宠相同, 仍使用式 (3.6)

此外还需区分 Rank 的两个使用场景: (1) 转生公式内部计算 F_x 时对原宠判 Rank, 其中首转使用普通表、二转使用转生表; (2) 转生完成后, 新成长率 $g^{(new)}$ 重新进入成长流程时统一按转生表判 Rank。二者所处阶段不同, 不宜混同。

3.2.4 升级阶段的随机增量

每次升级时系统再执行两步: 一次总和为 10 的随机加点 (不设单项限制), 以及从 Rank 对应区间中抽取倍率 $fRand$ 。若第 ℓ 级到第 $\ell + 1$ 级的随机加点记为 $p_i^{(\ell)}$, 则升级增量为

$$\Delta g_{r,i}^{(\ell)} = \left\lfloor (g_i^{(s)} + p_i^{(\ell)}) fRand^{(\ell)} \right\rfloor. \quad (3.10)$$

由式 (3.10) 可知, 高等级宠物的最终属性对应的是一个分布而非固定点; 成长模拟中输出分位数的做法, 正是对此一分布结构的数值呈现。

3.3 成长约束的两种来源

宠物系统中的上限可分为两类: 随机模型侧约束, 例如出生 10 点随机分配中“单项最多 4 点”的工具侧限制; 以及成长档压缩约束, 例如蛋喂药、孵化中的单项上限与总和上限。前者只影响初始点的离散分布, 后者则直接改变成长档本身并贯穿整个生命周期。养成优化的自然目标因此应是孵化后成长 h , 而非蛋基础成长或出生后的一级面板。

在这类带上限的整数系统中, 单项上限、总和上限与取整顺序都会在可行域上制造折角与跳点, 因而后续问题应理解为离散约束优化, 而非连续配比问题。

3.4 蛋基础成长与孵化后成长的区分

融合与喂药系统引入蛋基础成长与孵化后成长两个概念。蛋基础成长记为

$$\mathbf{b} = (b_V, b_S, b_T, b_D), \quad (3.11)$$

它表示融合完成后、喂药前或喂药后的蛋面数值；再记孵化后成长为

$$\mathbf{h} = (h_V, h_S, h_T, h_D), \quad (3.12)$$

它表示按孵化公式折算之后、但尚未进入出生后 10 点随机加点之前的成长档。当前工具中的优化问题和大多数比较均以 $\mathbf{h}$ 为对象，而不直接以一级显示属性为对象。

无喂药时，孵化是以 $\mathbf{b}$ 为输入、以 $\mathbf{h}$ 为输出的整数映射；有喂药时，还需经过药效折算、单项裁切、总和压缩和按比例回分配（详见下一章）。孵化后成长 $\mathbf{h}$ 是进入成长流程的成长档；出生后状态还将叠加 ± 2 波动、Rank 判定和出生随机加点。

综上，蛋面成长只是中间状态，经孵化映射后生成的 $\mathbf{h}$ 才是真正进入后续成长流程的成长档。本章所建立的“两层结构”，即成长档本身以及由成长档驱动的随机成长过程，将构成后续章节的共同基础。

第四章 宠物养成链：转生、融合与蛋喂养

转生、融合与蛋喂药在源码中分散于若干彼此衔接的模块，但从数值结构上看，它们共同构成了一条连续的宠物养成链。原宠既可以通过转生重构成长档，也可以通过融合生成蛋基础成长；蛋基础成长再经喂药与孵化映射为新的成长档，随后重新进入普通成长流程。本章据此将相关机制统一表述为一组可组合的离散变换，为后文分析高等级面板与战斗能力的形成路径奠定基础。

4.1 转生作为成长档重构

4.1.1 输入变量与整体流程

转生模块的输入包括原宠成长率 $\mathbf{g} = (g_1, g_2, g_3, g_4)$ 、MM 向量 $\mathbf{m} = (m_1, m_2, m_3, m_4)$ 、宠物等级 L_v 、转生次数以及转生能力上限 $A_{\max}$ （首转默认 150，二转默认 200，可人工覆盖）。

转生首先生成一组新的基础成长率，该成长率再经历普通出生与升级流程。记转生后的确定性成长率为 $\mathbf{g}^{(new)}$ ，则逻辑链可写为

$$(\mathbf{g}, \mathbf{m}, L_v, \text{转生次数}, A_{\max}) \longrightarrow A \longrightarrow \mathbf{g}^{(new)} \longrightarrow \text{普通成长流程}. \quad (4.1)$$

其中 A 即当前实现中的转生能力值。转生并不是对最终属性的直接修补，而是在成长档层面先完成一次离散重构，再经普通成长流程传导到高等级数值。

4.1.2 Rank、Fx 与转生能力值的形成

设原宠总和为

$$T_2 = \sum_{i=1}^4 g_i, \quad (4.2)$$

MM 总和经上限裁切后为

$$T_1 = \min \left\{ \sum_{i=1}^4 m_i, 150 \right\}. \quad (4.3)$$

首转时，原宠 Rank 按普通宠物的 Rank 表判定；二转时，则改按转生宠 Rank 表判定。记此 Rank 为 r ，则项目实现中的

$$F_x = \lfloor (5 - r) \times 1.2 \rfloor + 5. \quad (4.4)$$

将等级截到 130 以内，记

$$\tilde{L} = \min\{L_v, 130\}, \quad (4.5)$$

则转生能力值可整理为

$$A = \left\lfloor 1.3 \left(\frac{T_1}{100} \right)^5 \right\rfloor + T_2 + \left\lfloor \frac{\tilde{L} - 100}{F_x} \right\rfloor. \quad (4.6)$$

再施加转生能力上限，得到

$$A^* = \min\{A, A_{\max}\}. \quad (4.7)$$

式 (4.6)–(4.7) 表明，MM 总和通过五次方项提供高阶放大，而不是线性叠加；等级补正只在 100 级以上生效，并在 130 级后饱和；最终结果还受到 $A_{\max}$ 的上限约束。

表 4.1: 转生能力值参数扫描表

MM 总和	首转原始 A	首转封顶后	二转原始 A	二转封顶后
80	122	122	123	123
100	123	123	124	124
120	125	125	126	126
130	126	126	127	127
140	128	128	129	129
150	131	131	132	132

由于四维结果各自向下取整，分配后的四项总和一般小于 A^* ，产生 1 至 3 点的取整损失。表 4.2 的扫描显示，四维总和随 A^* 以台阶方式增长。

表 4.2: 转生能力值分配后的取整损失表

A^*	分配后四维	四维总和	取整损失
140	46/39/31/22	138	2
145	47/40/32/23	142	3
150	49/42/33/24	148	2
155	51/43/34/25	153	2
160	52/44/36/26	158	2

4.1.3 能力值向四维成长的分配

A^* 按“原宠四倍权重、MM 一倍权重”分配到四维。记总权重为

$$W = T_1 + 4T_2, \quad (4.8)$$

则每一维的新成长率为

$$g_i^{(new)} = \left\lfloor \frac{A^*(m_i + 4g_i)}{W} \right\rfloor, \quad i = 1, 2, 3, 4. \quad (4.9)$$

由于权重中原宠占四倍、MM 仅占一倍，MM 的作用更接近于对原结构的方向性修正，而非对原宠结构的完全替换。同时， $\mathbf{g}^{(new)}$ 仍将经历 ± 2 波动、Rank 判定、出生随机加点与升级随机，因此转生后的高等级属性依然应理解为分布而非单一点值。

4.2 融合作为逐步截断变换

4.2.1 主宠、副宠与低等级折扣次序

融合模块的输入为主宠与一至两只副宠的等级及成长率。等级低于 80 的参与宠需先逐项做低等级折扣：

$$g'_i = \lfloor 0.8g_i \rfloor. \quad (4.10)$$

关键在于低等级折扣先于后续贡献系数执行：每只低等级宠物先单独完成截断，再进入主宠贡献或副宠平均步骤。融合因此并非简单的线性平均，而是由多层取整算子串联而成的离散变换。

4.2.2 主宠贡献、副宠贡献与蛋基础成长

设主宠折扣后的成长向量为 $\mathbf{g}^{(M)}$ ，副宠折扣后的成长向量分别为 $\mathbf{g}^{(S1)}$ 、 $\mathbf{g}^{(S2)}$ 。若副宠数量为 $n_s \in \{1, 2\}$ ，则先求逐项和

$$s_i = \sum_{k=1}^{n_s} g_i^{(Sk)}, \quad (4.11)$$

再逐项做整数平均

$$\bar{s}_i = \left\lfloor \frac{s_i}{n_s} \right\rfloor. \quad (4.12)$$

随后，主宠贡献与副宠贡献分别为

$$c_i^{(M)} = \left\lfloor 0.6g_i^{(M)} \right\rfloor, \quad (4.13)$$

$$c_i^{(S)} = \lfloor 0.4\bar{s}_i \rfloor. \quad (4.14)$$

最终蛋基础成长为

$$b_i = c_i^{(M)} + c_i^{(S)}. \quad (4.15)$$

式 (4.12) 与式 (4.14) 的先后次序至关重要。“先平均再乘 0.4”与“先乘 0.4 再平均”在整数系统中并不等价。当前实现采用前者，因此融合模型应理解为次序不可交换的逐步取整算子。

表 4.3: 低等级副宠参与融合时的取整顺序对比

案例	副宠 1 (等级 / 四维)	副宠 2 (等级 / 四维)	gmsv 顺序	先平均后再做低级折扣 (误解)	受影响维度
A	60 级 / 20/20/20/20	70 级 / 20/20/25/25	30/27/25/22	30/27/24/21	T, D 各少 1
B	60 级 / 21/20/20/20	70 级 / 20/25/20/25	30/28/24/22	30/27/24/21	S, D 各少 1
C	60 级 / 25/20/20/20	70 级 / 20/29/20/20	31/28/24/21	30/28/24/21	V 少 1

表 4.3 中主宠统一固定为 80 级、成长 40/35/30/25。之所以专门列出这三组案例，是因为它们都能实际展示“先对低等级副宠逐只做 0.8 折扣，再平均”与“先平均，再整体做一次低等级折扣”之间的差异。换言之，融合中的取整顺序并非形式细节，而是足以改变蛋基础成长整数结果的实质性机制。

4.3 喂药与孵化的两级压缩结构

4.3.1 药效累加与工作值折算

蛋喂药的输入为蛋基础成长 $\mathbf{b}$ 与 4×5 喂药方案矩阵。记第 i 维在第 ℓ 级药上的颗数为 $c_{i\ell}$ ，第 ℓ 级药的原始值为 e_ℓ ，则药效原始和为

$$r_i = \sum_{\ell=1}^5 c_{i\ell} e_\ell. \quad (4.16)$$

默认 e_ℓ 依次为 25, 50, 75, 100, 125，总颗数固定为 40。药效经折算后为

$$u_i = \min \left\{ \left\lfloor \frac{7r_i}{1000} \right\rfloor, 60 \right\}. \quad (4.17)$$

折算单项上限为 60。不同等级药在原始值层面虽然可以线性相加，但在折算与单项上限进入后，最终结果会出现显著的离散差异。

4.3.2 工作值压缩与比例回分配

记折算后的总和为

$$U = \sum_{i=1}^4 u_i. \quad (4.18)$$

若 $U \leq 50$ ，则直接令 $w_i = u_i$ ；若 $U > 50$ ，则先做整体压缩：

$$w_i = \left\lfloor \frac{50u_i}{U} \right\rfloor. \quad (4.19)$$

w_i 再经“半折算 + 按比例回分配”生成孵化后成长。记

$$T_1 = \sum_{i=1}^4 w_i, \quad T_2 = \sum_{i=1}^4 b_i, \quad (4.20)$$

并定义

$$T = \left\lfloor \frac{T_1}{2} \right\rfloor + T_2. \quad (4.21)$$

当前实现对每一维使用

$$f_i = b_i + \left\lfloor \frac{w_i}{2} \right\rfloor, \quad (4.22)$$

再计算增量

$$\Delta_i = \left\lfloor \frac{f_i}{T} T_1 \right\rfloor, \quad (4.23)$$

从而得到初步孵化成长

$$h'_i = \text{clip}_{[1,60]}(b_i + \Delta_i). \quad (4.24)$$

式 (4.24) 中 $w_i/2$ 和 $T_1/2$ 均为整数除法，因此药效并入蛋成长时会产生额外的整数损失。

4.3.3 孵化总和压缩与最终成长档

记初步孵化成长总和为

$$H' = \sum_{i=1}^4 h'_i. \quad (4.25)$$

若 $H' \leq 150$ ，则最终孵化后成长直接取 $h_i = h'_i$ ；若 $H' > 150$ ，则再施加一次总和压缩：

$$h_i = \text{clip}_{[1,60]} \left(\left\lfloor \frac{150h'_i}{H'} \right\rfloor \right). \quad (4.26)$$

孵化后成长由式 (4.17)–(4.26) 共同决定，本质上是一个同时受到单项上限、总和上限、比例回分配和多次向下取整约束的组合优化问题。

在默认参数下，所有药的原始总和最多为 $40 \times 125 = 5000$ ，折算后四维工作值总和满足

$$\sum_i u_i \leq \left\lfloor \frac{7 \times 5000}{1000} \right\rfloor = 35, \quad (4.27)$$

由此可见，式 (4.19) 所表示的 50 压缩在默认规则下并不触发；真正起决定作用的总和约束，是最终孵化阶段的 150 总和压缩。

表 4.4: 默认 40 药规则下的孵化案例表

案例	蛋基础 b	喂药方案示意	折算工作值 u	是否触发 150 压缩	最终 h
混合药达 150	35/35/35/35	T 喂 10 颗四级药， D 喂 30 颗一级药	0/0/7/5	否	37/37/38/38
单堆五级被压缩	35/35/35/35	V 喂 40 颗五级药	35/0/0/0	是	40/36/36/36
敏捷集中五级药	30/30/30/30	D 喂 40 颗五级药	0/0/0/35	是	36/36/36/41
高基础敏捷集中	60/50/20/20	D 喂 40 颗五级药	0/0/0/35	是	52/52/21/23

表 4.4 所强调的是两类更具解释力的对照。第一行与第二行使用同一组基础蛋面 35/35/35/35：若把高低级药混合配置为“10 颗四级药 + 30 颗一级药”，最终可以在不触发 150 总和压缩的前提下恰好达到总和 150；但若把 40 颗五级药全部堆到单项，则虽然单项工作值更高，最终却因触发 150 压缩而只剩总和 148。第三、四行则说明：即使把五级药全部集中到敏捷，也不会触发式 (4.19) 所对应的 50 工作值压缩，真正起作用的仍是末端的 150 总和压缩及其后的整数回分配。

若进一步把问题聚焦到“单项最终能否到 60”，则还会出现另一层阈值现象：即使某一维的喂药全部集中，只要其余三项总和过高，最终 150 压缩仍会把该维重新压回去。表 4.5 给出一个固定实验口径：把 40 颗五级药全部喂给敏捷维，并让其余三项基础成长尽量均分。表中列出的最小基础值阈值，正可以用来说明“为什么蛋面不到某个点，孵化后单项就上不去 60”。

表 4.5: 默认 40 药规则下单项到达 60 的基础阈值表

其余三项总和	其余三项 基础示例	D 最小基础值	最终孵化四维	最终总和
30	10/10/10	38	14/14/14/60	102
45	15/15/15	41	20/20/20/60	120
60	20/20/20	43	25/25/25/60	135
75	25/25/25	54	30/30/30/60	150
90	30/30/30	—	—	—

表 4.5 的实验口径固定为“其余三项尽量均分，且 40 颗五级药全部投入 D ”。这一写法将“总和阈值”转写为更直观的整数案例：例如当其余三项已经是 25/25/25 时，若想让最终敏捷项到达 60，则基础 D 至少要到 54；而当其余三项已经是 30/30/30 时，无论基础 D 如何上调，在当前默认规则下都无法再把最终 D 压到 60 以上。

4.4 孵化结果的解释口径

孵化过程可分为两层：确定性孵化成长 h ，以及进入出生流程后再叠加 ± 2 波动、Rank 判定与 10 点随机加点的随机出生状态。本文将前者作为优化对象，从而把可控的喂药方案比较与出生阶段的随机实现明确区分开来。

4.5 养成流水线的统一表示

综合前述各节，本文将宠物养成系统写成如下流水线：

$$\begin{aligned}
 &\text{原宠} \xrightarrow{\text{转生或保留}} \text{成长档} \xrightarrow{\text{融合}} \text{蛋基础成长 } b \\
 &\quad \quad \quad \xrightarrow{\text{喂药与孵化}} \text{孵化后成长 } h \xrightarrow{\text{出生与升级随机}} \text{高等级面板}.
 \end{aligned} \tag{4.28}$$

这一流水线将原本分散于不同位置的机制统一为状态变换序列。各环节共同呈现出“先截断、再裁切、再按比例分配”的离散结构，因此宠物培养问题天然兼具离散优化与状态转移两重性质。

第五章 宠物养成到战斗能力的映射

前两章已经分别给出成长生成机制与养成链的离散结构。本章在此基础上，将二者收束为一条从养成输入到战斗输出的统一映射，并讨论成长率如何传导到攻击、防御与敏捷，转生、融合和喂药又如何改变这种传导关系。

5.1 从 (V, S, T, D) 到战斗输出的映射

宠物成长率并不会直接表现为玩家可见的战斗面板，而是要经由出生随机、升级随机与显示公式的共同作用，最终形成攻击、防御与敏捷。若把高等级时的原始累计点向量记为

$$\mathbf{r} = (V_r, S_r, T_r, D_r), \quad (5.1)$$

则第 3 章已经给出显示映射：生命主要由 V_r 主导，攻击主要由 S_r 主导，防御主要由 T_r 主导，敏捷则近乎由 D_r 单独决定。从边际影响上看，四项成长对战斗能力的主要路径可以概括为

$$V \rightarrow HP, \quad S \rightarrow ATK, \quad T \rightarrow DEF, \quad D \rightarrow SPD. \quad (5.2)$$

式 (5.2) 只刻画主路径。显示公式中仍存在交叉项： V 和 D 会少量进入攻击、防御， S 和 T 也会通过总和与随机路径间接影响成长评分。因而，常见的“主 V 宠”“主 D 宠”等表述，在源码层面更准确地说，应理解为“某一维在主路径上占优势”，而非其余三维完全不起作用。

进入实战后，显示属性还会继续影响速度与动作结果。其中，敏捷是最关键的桥梁变量：宠物显示敏捷越高，在先手、回避、会心与反击相关结果中越容易取得优势。成长中的 D 维因此并不只对应面板敏捷的增加，而是会沿着更长的判定链进一步影响先手权与概率结果。

5.2 从转生、融合到战斗能力的长期收益

转生、融合和蛋喂药均作用于“进入成长流程之前的成长档”，而非直接修改战斗公式的输入，因此其收益具有显著的长期累积性。设原始成长档为 $\mathbf{g}$ ，转生后成长档为 $\mathbf{g}^{(new)}$ ，融合蛋基础成长为 $\mathbf{b}$ ，喂药并孵化后的成长档为 $\mathbf{h}$ ，则战斗能力的比较实质上是

在比较

$$\Psi(\mathbf{g}), \quad \Psi(\mathbf{g}^{(new)}), \quad \Psi(\mathbf{h}), \quad (5.3)$$

其中 $\Psi(\cdot)$ 表示“成长随机 $\rightarrow$ 高等级显示属性 $\rightarrow$ 速度与动作结果 $\rightarrow$ 战斗结果”的复合映射。

转生的长期收益主要体现在两个方面。第一，它通过转生能力值和按权重重分配，整体改变了四维成长的起点；第二，转生后的新成长率进入的是“转生宠 Rank 表”，从而改变了后续升级随机的区间结构。转生因此同时改变总和、方向与 Rank 的解释规则。

融合和喂药的长期收益则更接近一个带约束的优化问题。融合先通过主宠 0.6 与副宠平均后 0.4 生成蛋基础成长，喂药再经过单项折算、按比例回分配与孵化总和压缩，最终得到孵化后成长 $\mathbf{h}$ 。融合链条的收益因此不仅取决于投入素材与药值的名义规模，还取决于这些数值在多次取整与上限约束之后实际保留下来的有效份额。

5.3 从蛋喂养到战斗目标的定向优化

喂药和孵化在进入普通成长流程之前确定一组新的成长档，因此，对战斗目标的优化首先应落在 $\mathbf{h}$ 上。若把一个战斗目标写成效用函数 U ，则蛋喂养问题可抽象为

$$\max_{\text{喂药方案}} U(\Psi(\mathbf{h})). \quad (5.4)$$

在实际工具中，为了降低求解复杂度，常先用一个线性代理目标

$$U_0(\mathbf{h}) = w_V h_V + w_S h_S + w_T h_T + w_D h_D \quad (5.5)$$

来近似真正的战斗目标。

式 (5.5) 的意义在于，它将复杂的战斗诉求先投影回孵化后成长空间。例如，若目标偏向先手与回避，则 w_D 应更高；若目标偏向骑宠近战爆发，则 w_S 与 w_D 通常应同时抬高；若目标偏向承伤与稳定性，则 w_V 与 w_T 更为重要。得到候选解之后，再将其带回更完整的战斗模型中作二次比较。

喂药阶段与最终战斗结果之间还隔着出生后随机和升级随机，若直接对最终战斗结果做全局枚举，计算代价会显著升高。对 $\mathbf{h}$ 先做代理优化，则可将这一高维随机问题拆解为“先确定方向，再在随机层验证”的两步过程。

5.4 宠物忠诚失控 (nono) 与动作改写

宠物成长除了通过面板三围进入战斗外，忠诚还会通过本回合固定忠诚值直接改变宠物最终执行的动作。忠诚检查位于“动作写入之后、动作分发之前”的阶段，因此它并不改变成长输入，却会改变成长最终如何体现为战斗输出。

“宠物成长 $\rightarrow$ 战斗输出”的链条因此不能仅写成属性映射。对低忠诚宠而言，战斗输出更适合表述为“原指令 $\rightarrow$ 忠诚检查 $\rightarrow$ 实际动作”的复合结果。

5.4.1 忠诚阈值与改写结果

设当前回合读取到的固定忠诚值为 a ，并记 $\text{RAND}(1, 100)$ 的结果为 R 。忠诚失控并不是单一阈值事件，而是按 a 的区间把宠物分到不同改写模式，如表 5.1 所示。

表 5.1: 宠物忠诚失控 (nono) 的动作改写区间

忠诚区间 a	触发条件	本回合结果
$a \geq 80$	恒不触发	完全按玩家原指令行动
$70 \leq a < 80$	$R < 10$	9% 概率进入“乱目标”
$60 \leq a < 70$	$R < 20$	19% 概率进入“乱目标”
$50 \leq a < 60$	$R < 35$	34% 概率进入“乱目标”
$40 \leq a < 50$	$R < 50$	49% 概率进入“乱目标”
$30 \leq a < 40$	$R < 70$	69% 概率改放另一项宠技
$20 \leq a < 30$	$R < 70$	69% 概率改放另一项宠技
$10 \leq a < 20$	$R < 80$	79% 概率攻击主人，否则攻击敌人
$0 \leq a < 10$	$R < 60$	59% 概率攻击主人，否则直接逃跑

表 5.1 中的概率之所以是 9%、19%、34% 而不是看上去更直觉的 10%、20%、35%，原因在于这里使用的是严格不等式 $<$ ，并且随机值来自闭区间整数 $\text{RAND}(1, 100)$ 。例如，条件 $R < 10$ 只会在 $R = 1, \dots, 9$ 时成立。

还需指出，只有 $20 \leq a < 40$ 这一段的“随机动作”模式，才会把玩家原本指定的宠技 A 改写成另一项宠技 O； $40 \leq a < 80$ 只会打乱目标， $0 \leq a < 20$ 则会进一步退化成攻击主人、攻击敌人或逃跑。因此，实战中“原本指定 A，却实际放出 O”的 nono 现象，主要集中在 $20 \leq a < 40$ 这一区间。

5.4.2 为何会出现“技能 A 的面板修正保留，但实际放出技能 O”

这一现象源于动作执行顺序。宠物技能在玩家输入阶段就会先写入一组本回合工作值，例如

(主动作编号, 目标槽, 动作附加参数,

战斗攻击, 战斗防御, 战斗速度, ...).

(5.6)

随后在战斗主循环中，系统先做状态序列与基础修正，再进行忠诚检查；只有在这一步之后，才按主动作编号进入具体动作分支。于是，若宠物先按玩家原指令 A 写入了攻、防、敏等工作值，而忠诚检查又把动作编号改成另一项技能 O，那么是否出现“先吃到 A 的面板效果，再放出 O”，就取决于 O 会不会把这些工作值重新覆盖。

这类情况在当前口径下确实存在。例如某些宠技会先把

战斗攻击 $\leftarrow 0.8 \times$ 固定力量, 战斗防御 $\leftarrow 0.7 \times$ 固定防御,

(5.7)

再写入对应动作；而另一些宠技只改写动作编号、目标和动作附加参数，并不重写原有的攻防工作值。于是，若宠物先选中了前者，但在忠诚失控的“随机动作”分支里又被改写成后者，就会出现“前一个技能留下的攻防修正仍在，最终放出的却是后一个技能”的连锁效果。

宠物战斗输出因此不能仅写成“技能 O 的伤害公式”。对低忠诚宠，更准确的表达应为

$$\text{最终输出} = \Omega(\text{原指令 } A, \text{忠诚区间}, \text{改写结果 } O, O \text{ 是否覆盖 } A \text{ 已写入的工作值}). \quad (5.8)$$

低忠诚宠的实战收益不仅由成长与技能池决定，还取决于本回合动作写入顺序以及各技能对工作槽的覆盖范围。

5.5 不同培养目标下的宠物类型

若把战斗目标按主导维度划分，可以把宠物大致分成四类。第一类是先手/控制型，核心在于尽可能抬高 D ，使宠物本体或骑宠混合后的速度敏更容易跨过乱敏阈值。第二类是近战爆发型，更重视 S 与 D 的组合： S 负责提高攻击路径， D 负责抢先手和提高会心、命中相关概率。第三类是承伤稳定型，其重点通常落在 V 与 T ，以换取更高血量和防御。第四类则是均衡型，在总和受限时尝试避免某一维过高导致整体压缩。

上述分类对应的是不同目标函数下的优化方向，而非彼此独立的自然种类。例如，同样一只高 D 宠，在骑宠近战白狼体系中可能被视为先手核心；在骑宠远程体系中，其价值则可能部分让位于人物本体速度。宠物类型的界定，最终仍取决于人物职业、武器、骑宠状态与战斗目标的联合作用。

第六章 离散优化模型、案例与策略含义

前文关于成长档、转生、融合、喂药与孵化的分析，共同指向一个高度离散、带多重上限且包含大量取整跳点的优化问题。前几章已经说明成长如何生成、压缩如何发生，以及这些成长差异如何映射到战斗输出。本章在此基础上将培养问题进一步表述为可计算对象，并结合代表性案例讨论其策略含义。

6.1 优化问题的形式化

将整个宠物培养过程写成统一映射后，可以把最终优化问题抽象为

$$\max_{x \in \mathcal{X}} U(\Psi(x)), \quad (6.1)$$

其中 x 表示决策变量， $\mathcal{X}$ 表示所有可行培养方案， Ψ 表示“培养方案 $\rightarrow$ 成长档 $\rightarrow$ 随机成长 $\rightarrow$ 战斗输出 $\rightarrow$ 战斗表现”的复合映射， U 则表示研究者真正关心的目标函数。

在当前口径下， x 至少可以包含三类变量：转生输入、融合输入和喂药方案。其中，喂药方案可优先形式化为整数规划。若记第 i 维第 ℓ 级药的颗数为 $c_{i\ell}$ ，则其约束满足

$$c_{i\ell} \in \mathbb{Z}_{\geq 0}, \quad \sum_{i=1}^4 \sum_{\ell=1}^5 c_{i\ell} = 40. \quad (6.2)$$

但由于后续还存在折算、单项上限、总和压缩和比例回分配，式 (6.2) 只是最外层约束，而不是完整可行域描述。

6.2 蛋喂养最优化

在喂药问题中，直接以最终战斗指标为目标，意味着必须把离散喂药、孵化、出生随机、升级随机和战斗公式全部纳入同一搜索，复杂度极高。本文因此采用两层处理：先在孵化后成长 $\mathbf{h}$ 上做离散优化，再把候选解带回更完整的战斗模型中验证。设蛋基础成长为 $\mathbf{b}$ ，喂药方案为 $\mathbf{c}$ ，则喂药层的中间问题可写为

$$\max_{\mathbf{c}} U_0(\mathbf{h}(\mathbf{c}; \mathbf{b})), \quad (6.3)$$

其中 U_0 可以是线性目标，也可以是词典序目标。

这一中间层必须显式建模，因为“全部喂 5 级药”并非总是最优。就默认 40 药规则而言，最终结果的主要约束来自末端 150 总和压缩，而非前段 50 工作值压缩；继续把药向同一维集中，往往只会把其余维度一并推入总和压缩区，从而无法继续提高目标维度的最终收益。在某些高基础蛋上，混入较低等级药反而可能优于全部使用五级药。这并非经验技巧，而是多次向下取整与总和上限共同作用下的离散最优化结论。

本文还引入了词典序目标，即先最大化主目标，再在主目标并列解中最大化次目标。其必要性来自大量退化解。例如，在药效原始值满足线性关系的情况下，不同药级组合可能给出完全相同的折算结果。若没有次级目标或其他偏好规则，输出候选解便会高度冗余。对更高等级素材的偏好，本质上是在这些退化解之间引入稳定的优先级。

6.3 战斗目标反推

实际关心的目标通常不是“令某个孵化后成长最大”，而是“令先手概率、命中率、会心率或骑宠伤害最大”。因此，需要将战斗目标反投影回成长空间。若记战斗目标为

$$U_{\text{battle}} = U(HP, ATK, DEF, SPD, p_{\text{hit}}, p_{\text{crit}}, \text{damage}, \dots), \quad (6.4)$$

则一个常见策略是先在当前点做局部线性化，把它近似为

$$U_{\text{battle}} \approx w_V h_V + w_S h_S + w_T h_T + w_D h_D + \text{const}, \quad (6.5)$$

再回到式 (6.3) 求解。

例如，若目标是提升玩家对玩家的先手与会心，则由于这两项都对敏捷高度敏感，线性化后通常会赋予 w_D 更高权重；若目标是骑宠近战爆发，则 w_S 与 w_D 往往同时较高，因为 S 主要影响攻击，而 D 同时影响先手和会心；若目标是承伤，则应提高 w_V 与 w_T 。“战斗目标 $\rightarrow$ 成长代理目标”的反推步骤，构成了各种喂药搜索模式背后的统一数学结构。

6.4 转生与融合策略比较

转生与融合属于不同层次的决策。前者重塑成长档后再交给完整成长随机，后者则把多只宠物的成长信息压缩进一个蛋面中间态。两者面临的约束瓶颈因此并不相同。

转生的主要瓶颈来自转生能力上限 A_{max} ，高端区间本质上体现为方向重分配问题。融合的主要瓶颈则来自逐层取整：低等级宠先做 0.8 折扣，再参与主副权重分配，最后蛋又要经过孵化压缩，损失沿途积累。

若当前已有高总和且方向一致的成熟宠，转生更适合保持原有方向并整体抬升；若目标是重组多只宠物的局部优势，则融合与喂药更适合用于方向重构。但一旦进入压缩区，无论转生还是融合，都不能再以“总和略高即一定更优”的线性直觉判断。

6.5 案例研究

下面给出三个不依赖具体服务器数值、但足以说明机制特征的案例。

6.5.1 案例一：先手优先的骑宠近战

设目标是让一只骑宠近战配置在 PVP 中尽量抢到先手，同时保留一定爆发。结合前章已经说明的战斗映射关系可知，骑宠近战时速度参照与攻击都更偏宠物，因此代理目标应更偏向 D 与 S 。相应的近似目标可写为

$$U_1 = \alpha h_D + \beta h_S, \quad \alpha > \beta > 0. \quad (6.6)$$

若基础蛋本来就有较高总和，则继续把所有高等级药堆到 D 上，可能因为触发总和压缩而连带损失 S ，从而在最终骑宠伤害上得不偿失。这正是词典序目标或多候选解比较有价值的场景。

6.5.2 案例二：高基础蛋的压缩陷阱

设一枚蛋的基础成长总和已经接近最终上限。若继续用“每一颗药都尽量给最高级”的策略，则孵化后成长很容易触发 150 总和压缩。此时，药效的边际收益不再等于原始药值。对这类问题，最优方案常常不是单纯堆单项，而是控制压缩前的分布，使目标维度在总和裁切后仍保留最大整数份额。离散最优解更接近“卡阈值”而非“无限堆高”。

6.5.3 案例三：主目标并列下的次目标选择

设若干喂药方案都能把主目标维度推到同一上限，此时单看主目标已经无法区分优劣。若没有次目标或偏好规则，输出会出现大量本质等价的退化解。本文采用“主目标 + 次目标 + 高等级素材优先”的方案，本质上是在一个大并列集合中引入稳定排序。这样做并不改变底层规则，而是提高输出结果的可解释性与可操作性。

综合以上案例可见，底层机制本身是一个离散、分段且带上限的系统，适合拆分为若干可验证的中间层，再逐层连接到战斗目标。实际分析时，可依次经过孵化后成长、高等级面板与战斗代理收益三层，以保持模型的可计算性与解释力。

第七章 结论

7.1 主要研究结论

本文围绕当前持有的服务端源码，重建了从宠物基础成长、转生、融合、蛋喂药与孵化到战斗输出的完整分析链。研究表明，宠物培养并不是若干独立规则的简单叠加，而是一条由成长随机、阶段性取整、总和压缩和输出映射共同组成的离散流程。沿着这一流程，可以稳定回答若干核心问题：高成长如何形成，转生与融合各自改变了什么，蛋喂药为何会出现明显的阈值效应，以及成长差异又如何转化为速度、攻防与动作稳定性的差异。

在此基础上，可以得到若干直接修正经验口径的结论。第一，转生不仅改变四维成长的方向分配，还会改变后续进入的 Rank 判定口径，因此其收益不能仅按“总和提升多少”衡量。第二，融合与喂药的关键并不只是素材或药值本身，而在于这些数值经过多次向下取整与总和压缩之后究竟保留了多少有效份额。第三，蛋喂药问题天然属于离散优化问题；在高基础蛋上，混合使用不同等级药以控制压缩边界，可能优于机械堆叠高级药。第四，宠物成长进入实战时并不是一步到位地转化成单一结果，而是先表现为显示面板，再沿不同路径影响速度与动作结果，因此同一成长维度的收益会在不同层面呈现不同效果。第五，忠诚失控并不改变成长输入，但会改写成长如何转化为本回合实际动作，因此低忠诚宠的实战输出必须连同动作改写顺序一起分析。

7.2 对培养经验的修正

本文修正的并非若干孤立的经验说法，而是若干长期沿用的分析方式。其一，宠物培养不宜继续按连续直觉处理，“总和更高必然更强”只在忽略压缩、取整与方向差异时才近似成立。其二，蛋喂药方案的优劣不应仅以药级高低判断，而应结合折算与压缩后的最终整数结果。其三，成长维度与战斗目标之间并不是一一等价的口号式对应，而是一条包含随机、映射与动作条件的复合链条。其四，低忠诚宠的收益评价不能只看面板与技能池，还必须纳入忠诚区间下的动作改写方式。

上述修正表明，经验性判断只有在给定变量口径与边界条件后，才具有稳定的解释力。对于一个长期依赖经验传播的老游戏系统而言，将经验整理为带条件的定量命题，本身就构成了必要的研究工作。

7.3 研究局限与后续工作

本文仍有若干明确局限。首先，全文口径以当前持有的服务端源码与当前编译路径为准，不同分支或后续版本可能改变个别实现细节。其次，本文虽然已经建立了较完整的宠物培养链，但在数值实验层面仍以结构分析与代表性案例为主，尚未扩展为覆盖全部素材空间的大规模枚举图谱。再次，成长到战斗输出的映射已经写清主路径，但更细粒度的职业、武器与战场配置差异仍可在后续研究中继续展开。

未来研究可沿三方面继续展开：其一，补充更系统的搜索实验，把本文的离散结论扩展为更完整的培养方案图谱；其二，将不同战斗目标下的代理目标进一步细化，使宠物培养与具体构筑之间建立更直接的量化联系；其三，继续核对少数仍需保守处理的边界分支，尤其是特殊宠技覆盖范围、低忠诚改写细节与不同编译口径下的差异。

总的来看，本文表明，只要回到源码、取整位置与状态入口本身，宠物培养这一看似经验性的系统仍然可以被整理为一套自洽、可复核且可计算的离散模型。这一模型既能解释长期积累的经验，也能指出这些经验的适用边界与修正条件。

附录 A 符号、术语与字段对照

本附录汇总正文采用的主要数学记号、核心中文术语及其与字段或动作入口之间的对应关系。表中仅保留宠物论文正文实际使用且含义唯一的主记号，以便在成长、蛋喂药与战斗映射三层之间建立稳定对应。

对象类别	正文记号 / 术语	源码字段 / 入口	说明
成长	V, S, T, D	抽象记号	宠物四项成长率，分别对应生命、攻击、防御、敏捷方向的成长来源
成长	g / 基础成长档	抽象记号	宠物基础成长的正文主记号；玩家口径中常写作 (V, S, T, D)
成长	(V, S, T, D) / 玩家口径别名	抽象记号	作为 g 的常用别名，本文不将其作为独立主记号
成长	$VSTD$ / 四项成长和	抽象记号	定义为 $V + S + T + D$
成长	b / 蛋基础成长	抽象记号	融合完成后、喂药前或喂药后的蛋面值
成长	h / 孵化后成长	抽象记号	按孵化规则折算后的成长，不含出生后额外随机点
成长	A / 转生能力值	工具内部旧称 <code>ans</code>	转生公式中的能力值主记号
成长	$A_{\max}$ / 转生能力上限	抽象记号	默认分析中对应首转 150、二转 200 的上限裁切
成长	L_v / 等级	抽象记号	用于区分一级、成长中、高等级与蛋阶段对象
映射	$r = (V_r, S_r, T_r, D_r)$	抽象记号	高等级阶段经过随机累计后的四维原始点

对象类别	正文记号 / 术语	源码字段 / 入口	说明
映射	HP, ATK, DEF, SPD	抽象记号	宠物显示面板中的生命、攻击、防御、敏捷
映射	Q / 速度相关输出量	CHAR_WORKQUICK	宠物行动顺序与速度相关结果的概括记号
映射	Ψ / 成长到输出的复合映射	抽象记号	表示成长、面板、速度与动作结果的综合映射
动作	主动作编号	CHAR_WORKBATTLECOM1	决定宠物最终进入哪一类动作分支的主编号槽
动作	动作附加参数	CHAR_WORKBATTLECOM3	高低位复用的动作参数槽, 可表示技能等级、子类型或额外修正
状态	a / 本回合固定忠诚值	CHAR_WORKFIXAI	宠物忠诚检查直接读取的本回合忠诚值
状态	宠物忠诚失控标志	CHAR_BATTLEFLG_AIBAD	表示该宠物本回合未按原指令行动的标志位
状态	Ω / 忠诚改写后的动作输出	抽象记号	表示原指令、忠诚区间与改写结果共同决定的最终动作

附录 B 公式总表

本附录汇总正文中的关键公式与约束条件，供读者按标签反向检索相应章节。表中的章号已经按宠物论文的独立结构重新编号。

编号	所在章节	含义
eq:pet-display-map	第 3 章	宠物原始累计点到显示 HP/ATK/DEF/SPD 的映射
eq:lv1-raw	第 3 章	一级初始化原始点数公式
eq:levelup-del-ta	第 3 章	升级时每一维原始点增长量
eq:trans-ans-raw	第 4 章	转生能力值原始公式
eq:trans-ans-cap	第 4 章	转生能力上限裁切
eq:trans-new-allloc	第 4 章	转生能力值按权重分配到四维成长
eq:fusion-level-discount	第 4 章	低于 80 级融合素材的逐项折扣
eq:sub-avg	第 4 章	副宠逐项平均的取整公式
eq:sub-contrib	第 4 章	融合时副宠贡献进入蛋成长的表达式
eq:egg-base	第 4 章	蛋基础成长生成公式
eq:feed-fold	第 4 章	蛋药原始值向工作值的折算公式
eq:work-cap	第 4 章	50 工作值总和压缩
eq:hatch-pre	第 4 章	孵化前的初步成长生成公式
eq:hatch-final	第 4 章	最终 150 总和压缩
eq:growth-main-path	第 5 章	四项成长到战斗输出主路径的概括映射
eq:linear-hatch-target	第 5 章	孵化后成长空间上的线性代理目标

编号	所在章节	含义
eq:global-opt	第 6 章	全局培养优化问题的统一形式
eq:feed-const raint	第 6 章	40 药约束
eq:hatch-opt	第 6 章	蛋喂药的中间优化问题

附录 C 源码索引

本附录列出正文中关键宠物机制所对应的源码文件、函数与已核对的核心入口，供后续追源、复核与对照使用。表中同时保留少量已经与源码对齐的本地工具实现条目，以便说明公式复核所依赖的辅助环境。

机制	文件 / 函数	关键位置	说明
普通宠 Rank 表	enemy.c / ENEMY_getRank	工具对齐	0 转宠 Rank 边界在工具中由 backend/simulator.py 对齐实现
转生宠 Rank 表	enemy.c / GetNewPet 内部	工具对齐	转生宠 Rank 边界在工具中由 backend/simulator.py 对齐实现
升级倍率表	char_data.c / RankRandTbl	工具对齐	各 Rank 对应的成长随机区间
成长模拟	tools/pet_revers e/backend/simula tor.py	全文件	对齐普通宠与转生宠的成长随机流程
转生计算	tools/pet_revers e/backend/simula tor.py / calc_t ransmigration	全文件	对齐转生能力值形成与权重分配规则
融合与孵化	tools/pet_revers e/backend/fusion .py、evolution.py	全文件	对齐融合成蛋、喂药折算与孵化压缩的整数规则
宠物忠诚失控	src/battle/battl e.c / BATTLE_Pet LoyalCheck	6802–6935	读取 CHAR_WORKFIXAI，按忠诚区间改写宠物本回合动作
随机宠技改写	src/battle/battl e.c / BATTLE_Pet RandomSkill	6708–6788	在随机动作模式下改放另一项宠技，并可能保留原技能已写入的工作值